

Discovery Executive Summary

Project: discovery-26-aug · Generated: 26/08/2026, 16:34:31

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Backend Modernization Analysis	—
2	Testing & Quality Assurance Analysis	—

1. Backend Modernization Analysis

EXECUTIVE SUMMARY

The PingCRM repository is a bifurcated codebase: a clean Laravel 11 CRM core (Contacts, Organizations, Users, Reports) sharing its scaffolding with a deeply problematic Legacy IVR Enterprise module that was bolted on in mid-2026. The Legacy IVR layer contains 12 "GodService" classes (one per module) each carrying 45 near-identical workflow methods that use PHP's `extract()` to materialize raw payload fields as local variables, hold a mutable static cache array, make unparameterized string-concatenated SQL via the `DB` facade, and `sleep(1)` on every call — representing over 540 unsafe `extract()` occurrences and 540 synchronous blocking seconds per full workflow cycle. Twelve parallel repository files compound this with explicit SQL-injection-vulnerable string-concatenated `LIKE` queries. The `config/ivr_legacy.php` config file commits a Salesforce client secret, plain-text password, and a master API key directly to source control, while all 12 GodService files each hardcode their own `LEGACY_IVR_KEY`. The generated legacy API surface accepts both `GET` and `POST` on all state-mutating endpoints without any authentication middleware, creating an unauthenticated write path to IVR data for any network-reachable client. Overall backend health is **High Risk**, driven primarily by widespread SQL injection, hundreds of hardcoded secrets, mass-assignment-open Eloquent models, missing authentication on the generated API surface, and synchronous blocking I/O in every legacy workflow.

§4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	4,940+ <code>extract()</code> calls across 50+ files	High Risk
H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	12 <code>public static \$sharedRuntimeCache</code> (one per GodService)	High Risk
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	~30% — 1,560 raw DB calls; controllers, traits, and services bypass repository	High Risk
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	5 pure-static helper classes (LegacyIvrCrypto with 80 methods, + 4 others)	High Risk
H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	70+ Ivr controllers contain inline SQL queries, direct service instantiation, and business logic	High Risk
H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	<10% — ~90 generated routes accept GET+POST; no versioning, no spec	High Risk
H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% — No OpenAPI spec, no API versioning, no contract tests	High Risk
H8	Weak Application Architecture	Modules following declared architecture %	>80%	50–80%	<50%	~40% — CRM core follows MVC; all 70+ IVR controllers are fat handlers with inline queries	High Risk

H9	Missing Module Inventory	Circular dependency count	0	1–3	>3	0 circular deps; Legacy and Http\Controllers\Ivr tightly coupled with no documented module API	Moderate
H10	Database Schema Weakness	FK indexes % + migrations with rollback %	Both >90%	One <90%	Both <90%	FK indexes ~60%; rollback: 100% (all migrations have <code>down()</code>)	Moderate
H11	Middleware Weakness	Required middleware present + ordered %	100%	80–99%	<80%	~55% — <code>/img/{path}</code> unprotected; entire generated ivr-legacy API has no auth middleware	High Risk
H12	Auth & Authorization Weakness	Protected routes guarded % + hashing algo	100% + bcrypt/argon2	One gap	Both bad	~60% routes guarded; bcrypt hashing (good); <code>\$proxies='*'</code> enables IP spoofing; <code>tenant_id=1</code> hardcoded	High Risk
H13	Backend Security Vulnerabilities	Injection + hardcoded secrets count	0 each	1–3 total	>3 total	SQL injection: 960 patterns; mass assignment: 12 models <code>\$guarded=[]</code> ; hardcoded secrets: 15+	High Risk
H14	Performance & Caching Gaps	N+1 patterns found	0	1–5	>5	540 <code>sleep(1)</code> blocking calls; 35+ N+1-prone model accessors; no caching layer	High Risk
H15	Outdated & Vulnerable Dependencies	Critical/High CVEs found	0	1–3	>3	<code>roave/security-advisories</code> present (CVE guard); no audit output; PHPStan at level 1	Moderate
H16	Secrets & Configuration in Source	Hardcoded secrets / .env committed	0	1–2	>2	15+ secrets: 12 <code>LEGACY_IVR_KEY</code> values + <code>config/ivr_legacy.php</code> master key, SF credentials, plain-text password	High Risk
H17	Backend Code Quality	Lint in CI + max cyclomatic complexity	Both good	One gap	Both bad	CI exists but PHPStan at level 1; GodService has 45 identical methods per class; extreme duplication	Moderate

§4.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Dynamic Variable Creation	Remove all <code>extract(\$payload)</code> calls; replace with typed Form Requests and explicit field mapping	High Risk	Critical
H2 — Global Mutable State	Remove <code>public static \$sharedRuntimeCache</code> from all 12 GodService classes; use per-request scoped instance caching	High Risk	Critical
H3 — Direct SQL Outside Data Layer	Move all controller and trait <code>DB::table()</code> / <code>DB::select()</code> calls into Repository classes	High Risk	Critical
H4 — Static / Singleton Abuse	Consolidate 80 <code>LegacyIvrCrypto::transformN()</code> into one parameterized method; convert helpers to injectable services	High Risk	High
H5 — Missing Service Layer	Extract business logic from 70+ IVR controllers into 12 dedicated <code>XxxService</code> classes with constructor DI	High Risk	Critical
H6 — API Sprawl	Replace <code>Route::match(['get','post'], ...)</code> with correct HTTP verbs; add <code>/api/v1/</code> versioning prefix	High Risk	High
H7 — Missing API Governance	Add <code>dedoc/scramble</code> for OpenAPI generation; add Spectral lint to CI; add contract tests	High Risk	High
H8 — Weak Application Architecture	Enforce thin-controller pattern; move all IVR queries to repositories; implement Laravel Policies	High Risk	High
H9 — Missing Module Inventory	Document module public APIs in <code>ARCHITECTURE.md</code> ; enforce module boundaries with PHPStan	Moderate	Medium
H10 — Database Schema Weakness	Add FK constraints and indexes to all IVR legacy tables	Moderate	Medium
H11 — Middleware Weakness	Add <code>->middleware('auth')</code> to <code>/img/{path}</code> ; wrap generated IVR API in <code>auth:sanctum</code> group	High Risk	Critical
H12 — Auth & Authorization Weakness	Restrict <code>\$proxies</code> to known load-balancer IPs; replace <code>\$tenantId = 1</code> with <code>Auth::user()->account_id</code> ; add Policies	High Risk	Critical
H13 — Backend Security Vulnerabilities	Parameterize all SQL in repositories; set explicit <code>\$fillable</code> on all IVR models; disable <code>allow_sql_debug</code>	High Risk	Critical
H14 — Performance & Caching Gaps	Remove all <code>sleep(1)</code> calls; dispatch remote syncs as queued jobs; add Redis caching; consolidate N+1 accessors	High Risk	High
H15 — Outdated & Vulnerable Dependencies	Run <code>composer audit</code> in CI on every PR; raise PHPStan to level 5; quarterly updates	Moderate	Medium
H16 — Secrets & Configuration in Source	Rotate all committed credentials immediately; move all to environment variables; add <code>git-secrets</code> pre-commit hook	High Risk	Critical

H17 — Backend Code Quality	Raise PHPStan to level 5; consolidate 45-method GodService and 40-method Repository classes; enforce in CI	Moderate	Medium
----------------------------	--	---	-------------------------------------

§4.5 Expected Outcomes

- Typed DTO / Form Request adoption** eliminates `extract()` -based variable shadowing — reducing injection-style risk from all 4,940+ dynamic variable sites to zero.
- Service layer introduction** enables all 12 IVR modules to share workflow logic across HTTP, CLI, and queue entry points — a single change to `AgentDeskService::store()` applies everywhere instead of 45+ separate methods.
- Parameterized SQL in repositories** eliminates the SQL injection attack surface across 960 vulnerable query sites and makes all data access independently testable.
- Authentication middleware on generated API** closes the unauthenticated write path to IVR data — currently requiring zero credentials from any network-reachable client.
- Credential rotation + secrets manager adoption** ensures all 15+ committed secrets are invalidated and replaced with short-lived, rotatable environment-scoped values.
- Redis caching layer** reduces database load on the hot `loadStats()` dashboard path by 90%+ for repeated loads within the same 30-second window.
- Removal of `sleep(1)` blocking calls** restores PHP-FPM worker throughput from 1 request/45 seconds per worker to normal Laravel concurrency — immediately improving responsiveness for all concurrent users.
- PHPStan raised to level 5+** turns the existing CI pipeline from cosmetic compliance to substantive quality enforcement, catching type mismatches and null-pointer dereferences before production.
{"stop_reason":"","end_turn":"","session_id":"f8d653d9-531e-4d45-beff-103ee10b302e","total_cost_usd":"2.2419829000000004","usage":{"input_tokens":38,"cache_creation_input_tokens":117154,"cache_read_input_tokens":3033353,"output_tokens":41039,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":117154,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":{"input_tokens":1,"output_tokens":3496,"cache_read_input_tokens":124987,"cache_creation_input_tokens":13354,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":13354},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":13274,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.013354,"contextWindow":200000,"maxOutputTokens":320000,"sonnet-4-6":{"inputTokens":38,"outputTokens":41039,"cacheReadInputTokens":3033353,"cacheCreationInputTokens":117154,"webSearchRequests":0,"costUSD":2.2286289000000004,"contextWindow":200000},"terminal_reason":"","completed"},"fast_mode_state":"off","uuid":"6d4745b4-1c11-49d4-8755-7ec817daaff7"}

2. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

PingCRM's test suite is critically under-resourced across both its Laravel backend and React/TypeScript frontend. Of 141 PHP source files and 903 frontend TypeScript/TSX files, only 5 test files exist in total — two real backend feature tests (Contacts, Organizations), one PHP unit placeholder that asserts `assertTrue(true)`, one frontend smoke test that asserts `expect(true).toBe(true)`, and a shared TestCase base class. Estimated overall coverage is well below 5%. Entire high-risk domains ship with zero test protection: the authentication flow, the Users module, Reports, and the entire IVR platform (83 controllers, 12 legacy GodServices, 12 repositories); no contract tests exist for any API endpoint or JSON response schema. Frontend Vitest tests are absent from the CI pipeline entirely, meaning the React layer has no automated regression gate. The two real backend feature tests run on CI but no evidence of being a required status check was found, compounding the risk.

§5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests (target 0)	0	1–3	>3	Auth, Users, Reports, IVR (83 controllers), 12 Legacy GodServices, 12 Repositories — >10 modules	High Risk
H2	Low Test Coverage	Overall coverage % (target >80%)	>80%	50–80%	<50%	Backend ~2% · Frontend ~0% (test-to-source ratio estimate — no coverage report present)	High Risk
H3	Missing Integration Tests	Key service/data boundaries covered % (target >70%)	>70%	30–70%	<30%	2 of ~30+ real service/controller boundaries = ~7%	High Risk
H4	Missing Contract Tests	Public APIs/contracts with contract tests % (target >80%)	>80%	40–80%	<40%	0 contract tests for any endpoint	High Risk
H5	Flaky / Skipped Tests	Skipped/flaky test count (target 0)	0	1–5	>5	0	Good
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	PHP tests run on PR/push; frontend Vitest absent from CI entirely	<span class="rating

							rating-high-risk">High Risk
H7	Trivial / Assertion-Free Tests (additional)	Tests with no meaningful assertion (target 0). Good: 0 · Moderate: 1 · High Risk: >1	0	1	>1	2 vanity tests (<code>ExampleTest.php</code> , <code>smoke.test.ts</code>) with <code>assertTrue(true)</code> / <code>expect(true).toBe(true)</code>	High Risk

§5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Write PHPUnit feature tests for <code>AuthenticatedSessionController</code> (login, logout, rate-limiting), <code>UsersController</code> (CRUD, owner flag), and at least one Index/Store test per IVR module (12 modules × 2 = 24 tests)	High Risk	Critical
H2 — Low Test Coverage	Enable PHPUnit coverage reporting in CI; set 30% initial gate in <code>phpunit.xml</code> ; configure <code>@vitepress/coverage-v8</code> with a 20% branch threshold for the frontend	High Risk	Critical
H3 — Missing Integration Tests	Extend <code>RefreshDatabase</code> feature tests to <code>UsersController</code> mutations and all IVR Index HTTP endpoints; add <code>LoginRequest</code> rate-limiter boundary test	High Risk	High
H4 — Missing Contract Tests	Add <code>AssertableInertia</code> prop-contract tests for all IVR Index pages asserting <code>rows</code> , <code>filters</code> , and <code>legacyMeta</code> keys; add JSON response shape assertions for <code>wantsJson()</code> branches in all 12 IVR modules	High Risk	High
H6 — No CI Test Gate (Frontend)	Add <code>npm test</code> (Vitest) and <code>npm tsc --noEmit</code> steps to <code>.github/workflows/tests.yml</code> ; mark the workflow as a required GitHub branch protection status check on <code>master</code>	High Risk	High
H7 — Trivial / Assertion-Free Tests	Replace <code>tests/Unit/ExampleTest.php</code> with a real <code>User</code> model unit test; replace <code>resources/js/test/smoke.test.ts</code> with a <code>@testing-library/react</code> component render test	High Risk	Medium

§5.5 Expected Outcomes

- Authentication and user-management regressions are caught by automated tests before they reach production, eliminating the risk of silent privilege escalation or account lockout.
- The entire IVR platform (83 controllers, 12 GodServices) has a regression safety net in place before any modernization or refactoring begins, allowing teams to refactor with confidence.
- Frontend Vitest and TypeScript checks run on every CI build, ensuring broken React components or type errors are caught at pull-request time rather than at runtime.
- API and Inertia prop-contract tests prevent breaking changes to JSON response shapes from silently corrupting IVR frontend pages after any backend change.
- A clearly visible, enforced coverage gate (30% initial, rising to 75%) gives the team a measurable quality trend and blocks coverage regressions from merging to `master` .", "stop_reason": "end_turn", "session_id": "5341f125-6bcb-491c-951f-dfa4b663448a", "total_cost_usd": 1.3179066999999998, "usage": {"input_tokens": 27, "cache_creation_input_tokens": 78709, "cache_read_input_tokens": 1615969, "output_tokens": 23360, "server_tool_use": {"web_search_requests": 0, "web_fetch_requests": 0}, "service_tier": "standard", "cache_creation": {"ephemeral_1h_input_tokens": 78709, "ephemeral_5m_input_tokens": 0}, "inference_geo": "not_available", "iterations": [{"input_tokens": 1, "output_tokens": 1832, "cache_read_input_tokens": 99363, "cache_creation_input_tokens": 533, "cache_creation": {"ephemeral_5m_input_tokens": 0, "ephemeral_1h_input_tokens": 533}, "type": "message"}], "speed": "standard", "modelUsage": [{"usage": "claude-haiku-4-5-20251001": {"inputTokens": 10301, "outputTokens": 16, "cacheReadInputTokens": 0, "cacheCreationInputTokens": 0, "webSearchRequests": 0, "costUSD": 0.010381, "contextWindow": 200000, "maxOutputTokens": 32000, "sonnet-4-6": {"inputTokens": 27, "outputTokens": 23360, "cacheReadInputTokens": 1615969, "cacheCreationInputTokens": 78709, "webSearchRequests": 0, "costUSD": 1.3075256999999998, "contextWindow": 200000, "terminal_reason": "completed", "fast_mode_state": "off", "uuid": "f45bcf14-e71e-4c25-8b71-12cf90f5c409"}}